

Day 10 – DSA Core II

Two Pointers, Sliding Window, Stacks & Queues

These patterns are important because they help replace slow nested loops with efficient one-pass solutions.

A useful mental model:

Two pointers	→ Track two positions in the same data
Sliding window	→ Track one continuous range
Stack	→ Last item added is processed first
Queue	→ First item added is processed first

1. Two Pointers

Concept

The two-pointer pattern uses two indexes to examine an array or string.

Instead of checking every possible pair using two nested loops, we move the pointers intelligently.

Array: [1, 2, 4, 6, 9]

L	R
↓	↓
[1, 2, 4, 6, 9]	

The two common versions are:

1. Opposite-direction pointers

- One starts from the left.
- One starts from the right.
- They move inward.

1. Same-direction pointers

- Both start near the beginning.
- One may represent a slow pointer.
- The other may represent a fast pointer.

How to Recognize Two Pointers

Think about two pointers when:

- The array is sorted.
- The question asks about a pair of elements.
- You need to compare values from both ends.
- You need to remove duplicates in place.
- You need to partition or rearrange an array.

- You need to check whether a string is a palindrome.

Typical phrases:

```
Find two numbers...
Sorted array...
Remove duplicates...
Move zeroes...
Palindrome...
Merge two sorted arrays...
```

2. Two Sum in a Sorted Array

Problem

Given a sorted array and a target, return the indexes of two numbers whose sum equals the target.

```
numbers = [1, 2, 4, 6, 9]
target = 10
```

The answer is $1 + 9 = 10$.

Brute-Force Approach

Check every possible pair:

```
1 with 2, 4, 6, 9
2 with 4, 6, 9
4 with 6, 9
...
```

Complexity:

- Time: $O(n^2)$
- Space: $O(1)$

Two-Pointer Intuition

Because the array is sorted:

- If the current sum is too small, move the left pointer right.
- If the current sum is too large, move the right pointer left.
- If the sum equals the target, return the answer.

```
[1, 2, 4, 6, 9]
 L           R

1 + 9 = 10
Answer found
```

Another example:

[1, 2, 4, 6, 9]
L R

Target = 8

$1 + 9 = 10 \rightarrow$ too large

Move R left:

[1, 2, 4, 6, 9]
L R

$1 + 6 = 7 \rightarrow$ too small

Move L right:

[1, 2, 4, 6, 9]
 L R

$2 + 6 = 8 \rightarrow$ found

Python Solution

```
from typing import List, Optional, Tuple

def two_sum_sorted(
    numbers: List[int],
    target: int,
) -> Optional[Tuple[int, int]]:
    """
    Return the indexes of two numbers whose sum equals target.

    The input array must be sorted in ascending order.
    """

    # Start one pointer at each end of the array.
    left = 0
    right = len(numbers) - 1

    # Continue while the pointers have not crossed.
    while left < right:
        current_sum = numbers[left] + numbers[right]

        if current_sum == target:
            return left, right

        if current_sum < target:
            # The sum is too small.
            # Move left forward to get a larger value.
            left += 1
        else:
            # The sum is too large.
            # Move right backward to get a smaller value.
            right -= 1

    # No valid pair exists.
    return None

numbers = [1, 2, 4, 6, 9]
print(two_sum_sorted(numbers, 8))
```

Output:

```
(1, 3)
```

Because:

```
numbers[1] + numbers[3]
= 2 + 6
= 8
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Tricky Part

Use:

```
while left < right:
```

Not:

```
while left <= right:
```

A pair must contain two different positions. When `left == right`, both pointers refer to the same element.

3. Same-Direction Two Pointers

Concept

In this version, both pointers move from left to right.

A common mental model is:

```
slow pointer → where the next valid result should be placed  
fast pointer → searches for valid elements
```

Example: Remove Duplicates from a Sorted Array

Input:

```
[1, 1, 2, 2, 3]
```

Result:

```
[1, 2, 3]
```

The fast pointer scans every value. The slow pointer tracks the last unique value.

```
[1, 1, 2, 2, 3]  
S  F
```

When the fast pointer finds a new value, copy it after the slow pointer.

Python Solution

```
from typing import List

def remove_duplicates(numbers: List[int]) -> int:
    """
    Remove duplicates from a sorted array in place.

    Returns the number of unique elements.
    The first `unique_count` positions contain the result.
    """

    if not numbers:
        return 0

    # slow points to the last unique element.
    slow = 0

    # fast checks every element after the first one.
    for fast in range(1, len(numbers)):
        if numbers[fast] != numbers[slow]:
            # A new unique value was found.
            slow += 1

            # Store it in the next result position.
            numbers[slow] = numbers[fast]

    # slow is an index, so the count is slow + 1.
    return slow + 1

numbers = [1, 1, 2, 2, 3]
unique_count = remove_duplicates(numbers)

print(unique_count)
print(numbers[:unique_count])
```

Output:

```
3
[1, 2, 3]
```

Complexity

- Time: $O(n)$
- Space: $O(1)$

Common Pitfall

`slow` is an index, not a count.

Therefore:

```
unique_count = slow + 1
```

Practical Use Cases

Two pointers can appear in:

- Comparing two sorted event streams
- Merging timestamped logs
- Removing duplicate document IDs
- Matching predicted labels with sorted ground-truth labels
- Comparing two token sequences
- Detecting palindrome-like input patterns

4. Sliding Window

Concept

A sliding window represents a continuous part of an array or string.

Instead of recalculating every range from scratch, we update the current range as it moves.

```
Array: [2, 4, 1, 5, 3]
```

```
Window size = 3
```

```
[2, 4, 1] 5 3  
2 [4, 1, 5] 3  
2 4 [1, 5, 3]
```

There are two main types:

1. Fixed-size window
2. Variable-size window

5. Fixed-Size Sliding Window

When to Use It

Use a fixed-size window when the problem asks about exactly k consecutive elements.

Typical phrases:

```
Maximum sum of k elements  
Average of every k values  
Requests during each 5-minute period  
Tokens in every chunk of size k
```

Example: Maximum Sum of k Consecutive Elements

```
numbers = [2, 1, 5, 1, 3, 2]
k = 3
```

Possible windows:

```
[2, 1, 5] → sum = 8
[1, 5, 1] → sum = 7
[5, 1, 3] → sum = 9
[1, 3, 2] → sum = 6
```

Answer: 9

Inefficient Approach

For every starting position, calculate the complete sum again.

Complexity:

```
Number of windows × work per window
O(n) × O(k)
= O(nk)
```

Efficient Sliding Window

Start with the first window:

```
[2, 1, 5] → 8
```

To move the window:

- Remove the element leaving from the left.
- Add the new element entering from the right.

```
Old window: [2, 1, 5]
```

```
Remove 2
```

```
Add 1
```

```
New window: [1, 5, 1]
```

Formula:

```
new_sum = old_sum - outgoing_value + incoming_value
```

Python Solution

```
from typing import List

def max_sum_of_size_k(numbers: List[int], k: int) -> int:
    """
    Return the maximum sum of exactly k consecutive elements.
    """

    if k <= 0:
        raise ValueError("k must be greater than zero")

    if k > len(numbers):
        raise ValueError("k cannot be larger than the array")

    # Build the first window containing indexes 0 through k - 1.
    window_sum = sum(numbers[:k])
    max_sum = window_sum

    # right is the index of the new element entering the window.
    for right in range(k, len(numbers)):
        # The outgoing element is exactly k positions behind right.
        outgoing_index = right - k

        window_sum -= numbers[outgoing_index]
        window_sum += numbers[right]

        max_sum = max(max_sum, window_sum)

    return max_sum

numbers = [2, 1, 5, 1, 3, 2]
print(max_sum_of_size_k(numbers, 3))
```

Output:

9

Complexity

- Time: $O(n)$
- Space: $O(1)$

Tricky Window Boundary

When right enters the window, the element leaving is:

$\text{right} - k$

For example, when:

```
right = 3
k = 3
```

The outgoing index is:

```
3 - 3 = 0
```

6. Variable-Size Sliding Window

Concept

A variable window expands and shrinks depending on a condition.

```
left           right
  ↓             ↓
[a, b, c, d, e, f]

Expand right → include more elements
Move left    → remove elements
```

General process:

1. Move right to expand the window.
2. Update information about the window.
3. While the window is invalid, move left.
4. Record the best valid window.

When to Use It

Typical problem phrases:

```
Longest substring with...
Shortest subarray with...
At most k distinct values...
Without repeating characters...
Minimum window containing...
Consecutive elements whose sum...
```

The important clue is that the answer is a **continuous substring or subarray**.

7. Longest Substring Without Repeating Characters

Problem

Given a string, find the length of the longest substring without duplicate characters.

Input: "abcabcbb"
Output: 3

Valid answer:

"abc"

Intuition

Start with an empty window.

String: a b c a b c b b
 L
 R

Expand right:

[a] b c a b c b b

Then:

[a b] c a b c b b

Then:

[a b c] a b c b b

When the next a enters, the window becomes invalid because a already exists.

[a b c a]

Move left until the duplicate is removed.

a [b c a]

Now the window is valid again.

Python Solution

```
def longest_unique_substring(text: str) -> int:
    """
    Return the length of the longest substring
    containing no repeated characters.
    """

    # Contains the characters currently inside the window.
    window_characters: set[str] = set()

    left = 0
    longest = 0

    # right expands the window one character at a time.
    for right in range(len(text)):
        current_character = text[right]

        # If the new character is already in the window,
        # shrink the window from the left.
        #
        # We use while instead of if because we may need
        # to remove multiple characters before the
        # duplicate is removed.
        while current_character in window_characters:
            window_characters.remove(text[left])
            left += 1

        # The current character is now safe to add.
        window_characters.add(current_character)

        # Both left and right are included in the window.
        # Therefore, length = right - left + 1.
        current_length = right - left + 1
        longest = max(longest, current_length)

    return longest

print(longest_unique_substring("abcabcbb"))
```

Output:

3

Complexity

- Time: $O(n)$
- Space: $O(\min(n, \text{number of possible characters}))$

Even though there is a while loop inside the for loop, each character enters and leaves the window at most once.

Therefore, the total work is still $O(n)$.

Tricky Parts

1. Window Length

Both endpoints are included:

```
window_length = right - left + 1
```

Example:

```
left = 2
right = 4

Indexes: 2, 3, 4
Length: 3
```

Calculation:

```
4 - 2 + 1 = 3
```

2. Use while, Not if

Incorrect:

```
if current_character in window_characters:
    ...
```

Correct:

```
while current_character in window_characters:
    ...
```

You may need to remove several characters before the duplicate disappears.

8. Shortest Subarray with Sum at Least Target

This is another variable-size pattern.

It works directly when the array contains positive numbers.

Example

```
numbers = [2, 3, 1, 2, 4, 3]
target = 7
```

The shortest valid subarray is:

```
[4, 3]
```

Length: 2

Python Solution

```
from typing import List

def minimum_subarray_length(target: int, numbers: List[int]) -> int:
    """
    Find the shortest continuous subarray whose sum
    is greater than or equal to target.

    Assumption: all numbers are positive.
    """

    left = 0
    window_sum = 0

    # Start with infinity so any valid length is smaller.
    minimum_length = float("inf")

    for right in range(len(numbers)):
        # Expand the window by adding the new right element.
        window_sum += numbers[right]

        # Once the window is valid, try making it smaller.
        while window_sum >= target:
            current_length = right - left + 1
            minimum_length = min(minimum_length, current_length)

            # Remove the leftmost value before moving left.
            window_sum -= numbers[left]
            left += 1

    if minimum_length == float("inf"):
        return 0

    return int(minimum_length)

numbers = [2, 3, 1, 2, 4, 3]
print(minimum_subarray_length(7, numbers))
```

Output:

2

Complexity

- Time: $O(n)$
- Space: $O(1)$

Important Limitation

This direct sliding-window logic depends on positive numbers.

With negative values, removing the leftmost element may increase or decrease the sum unpredictably. Such problems may require prefix sums, a deque, or another technique.

9. Sliding Window in Practical Systems

Log Processing

Question:

| What is the maximum number of errors in any five-minute period?

A fixed-size or time-based sliding window can maintain recent log events.

```
10:00 error
10:01 success
10:02 error
10:04 error
10:07 error
```

As time moves forward, remove events older than five minutes.

Rate Limiting

Maintain requests received during the latest time interval.

```
Window: last 60 seconds
Limit: 100 requests
```

When a request arrives:

1. Remove timestamps older than 60 seconds.
 2. Count remaining timestamps.
 3. Accept or reject the request.
-

Token and Stream Processing

Sliding windows can be used for:

- Splitting documents into overlapping chunks
 - Monitoring token usage over recent requests
 - Detecting repeated sequences in model output
 - Tracking recent failures in an inference service
 - Calculating moving averages for latency
-

10. Stack

Concept

A stack follows:

LIFO = Last In, First Out

Think of a stack of plates:

Push plate A
Push plate B
Push plate C

Top

↓

C

B

A

Pop → C is removed first

Common operations:

Operation	Meaning	Complexity
push	Add an item to the top	$O(1)$
pop	Remove the top item	$O(1)$
peek	Read the top item	$O(1)$
is_empty	Check whether empty	$O(1)$

In Python, a list can be used as a stack:

```
stack = []

stack.append("A") # Push
stack.append("B")

top = stack[-1]   # Peek
item = stack.pop() # Pop
```

Avoid removing from the beginning of a list for stack behavior.

How to Recognize Stack Problems

Look for:

- Nested structures
- Matching opening and closing symbols
- Undo operations
- Backtracking
- Previous greater or smaller element
- Expression evaluation
- Function call history
- Depth-first traversal

Typical phrases:

Balanced brackets
Valid parentheses
Undo
Nested
Most recent unmatched...
Next greater element
Previous smaller element

11. Balanced Parentheses

Problem

Determine whether brackets are correctly matched.

Valid:

```
() [] {}  
{ [] }
```

Invalid:

```
()  
( )]  
((
```

Intuition

When an opening bracket appears, push it onto the stack.

```
Input: ( { [ ] } )  
  
Read (  
Stack: (  
  
Read {  
Stack: ( {  
  
Read [  
Stack: ( { [  

```

When a closing bracket appears, it must match the top opening bracket.

```
Read ]
Top is [ → match

Read }
Top is { → match

Read )
Top is ( → match
```

At the end, the stack must be empty.

Python Solution

```
def is_valid_parentheses(text: str) -> bool:
    """
    Return True when every bracket is properly matched
    and nested.
    """

    stack: list[str] = []

    # Map each closing bracket to the opening bracket
    # it expects at the top of the stack.
    matching_opening = {
        ")": "(",
        "]": "[",
        "}": "{",
    }

    for character in text:
        if character in "([{":
            # Opening brackets wait for a future match.
            stack.append(character)

        elif character in matching_opening:
            # A closing bracket cannot be valid if
            # there is no opening bracket available.
            if not stack:
                return False

            most_recent_opening = stack.pop()

            if most_recent_opening != matching_opening[character]:
                return False

    # Any remaining opening bracket is unmatched.
    return len(stack) == 0

print(is_valid_parentheses("({[]})"))
print(is_valid_parentheses("([)]"))
```

Output:

```
True
False
```

Complexity

- Time: $O(n)$
- Space: $O(n)$ in the worst case

Common Pitfalls

Popping an empty stack

Incorrect:

```
opening = stack.pop()
```

Always check:

```
if not stack:
    return False
```

Forgetting remaining opening brackets

Input:

```
(( (
```

No closing bracket causes a mismatch during the loop, but the final stack is not empty.

Therefore, always finish with:

```
return len(stack) == 0
```

Practical Stack Use Cases

- Validating JSON-like nested input
- Parsing expressions
- Tracking nested function calls
- Undoing editor operations
- Depth-first search
- Evaluating configuration templates
- Validating generated code structures

12. Queue

Concept

A queue follows:

FIFO = First In, First Out

Think of people waiting in a line:

```

Front                                Back
  ↓                                ↓
[Alice, Bob, Charlie, David]

Alice entered first.
Alice leaves first.

```

Alice leaves first.

Common operations:

Operation	Meaning	Complexity
Enqueue	Add to the back	$O(1)$
Dequeue	Remove from the front	$O(1)$
Peek	Read the front	$O(1)$

Python Queue

Use `collections.deque`.

```
from collections import deque

request_queue = deque()

# Add items to the back.
request_queue.append("request-1")
request_queue.append("request-2")
request_queue.append("request-3")

# Remove the oldest item from the front.
oldest_request = request_queue.popleft()

print(oldest_request)
```

Output:

```
request-1
```

Why Not Use `list.pop(0)`?

```
items.pop(0)
```

This is $O(n)$ because every remaining item must be shifted left.

Use:

```
queue.popleft()
```

which is approximately $O(1)$.

How to Recognize Queue Problems

Look for:

- First-come-first-served processing
- Breadth-first search
- Level-by-level traversal
- Task scheduling
- Event processing
- Recent-item tracking
- Producer-consumer systems

Typical phrases:

```
Process in arrival order
Level by level
Minimum number of steps
Nearest...
Shortest path in an unweighted graph
Waiting jobs
```

Practical Queue Example: Request Processing

```
from collections import deque

def process_requests(requests: list[str]) -> None:
    """
    Process requests in the same order they arrived.
    """
    queue = deque(requests)

    while queue:
        # popleft removes the oldest request.
        current_request = queue.popleft()
        print(f"Processing: {current_request}")

process_requests([
    "embedding-job",
    "retrieval-job",
    "generation-job",
])
```

Output:

Processing: embedding-job
Processing: retrieval-job
Processing: generation-job

Practical Queue Use Cases

- API request buffering
- Asynchronous job processing
- Kafka consumer-style event processing
- Breadth-first search
- Message queues
- Model inference batch scheduling
- Processing document-ingestion jobs
- Tracking recent requests for rate limiting

13. Stack vs Queue

Feature	Stack	Queue
Rule	Last in, first out	First in, first out
Add operation	Push to top	Add to back
Remove operation	Pop from top	Remove from front
Common Python structure	<code>list</code>	<code>deque</code>
Common use	Nested structures, undo	Scheduling, BFS
Example	Browser back history	Request queue

Memory trick:

Stack → newest item gets priority
Queue → oldest item gets priority

14. Monotonic Stack

Concept

A monotonic stack keeps elements in a particular order.

It can be:

- Increasing
- Decreasing

For example, a decreasing stack may look like:

Bottom → [9, 7, 5, 2] ← Top

When a larger value arrives, smaller values may be removed.

Why Use It?

It helps answer questions such as:

- What is the next greater element?
- What is the previous smaller element?
- How many days until a warmer temperature?
- For how long does a price remain valid?
- What is the largest rectangle in a histogram?

A brute-force method may take $O(n^2)$. A monotonic stack often reduces this to $O(n)$.

15. Next Greater Element

Problem

For every element, find the first greater value appearing to its right.

Input:

[2, 1, 4, 3]

Output:

2 → 4
1 → 4
4 → none
3 → none

Result:

[4, 4, -1, -1]

Intuition

The stack stores indexes whose next greater element has not yet been found.

numbers = [2, 1, 4, 3]

Read 2:

Stack: [index of 2]

Read 1:

1 is not greater than 2
Stack: [2, 1]

Read 4:

4 > 1 → next greater for 1 is 4
4 > 2 → next greater for 2 is 4

Read 3:

3 is not greater than 4

Indexes left in the stack have no greater value on their right.

Python Solution

```
from typing import List

def next_greater_elements(numbers: List[int]) -> List[int]:
    """
    For every element, return the first greater element
    appearing to its right.

    Return -1 when no greater element exists.
    """

    result = [-1] * len(numbers)

    # Store indexes, not values.
    #
    # Indexes let us update the correct position
    # in the result array.
    stack: list[int] = []

    for current_index, current_value in enumerate(numbers):
        # Resolve all previous smaller values.
        while (
            stack
            and numbers[stack[-1]] < current_value
        ):
            previous_index = stack.pop()
            result[previous_index] = current_value

        # The current value now waits for a future
        # greater value.
        stack.append(current_index)

    return result

numbers = [2, 1, 4, 3]
print(next_greater_elements(numbers))
```

Output:

```
[4, 4, -1, -1]
```

Complexity

- Time: $O(n)$
- Space: $O(n)$

Although the code has a loop inside a loop, each index is:

- Pushed once
- Popped at most once

Therefore, total operations are linear.

Tricky Parts

Store Indexes

Storing indexes lets us do:

```
result[previous_index] = current_value
```

If only values were stored, handling duplicates and updating result positions would be difficult.

Greater vs Greater-or-Equal

This condition finds a strictly greater value:

```
numbers[stack[-1]] < current_value
```

Changing it to `<=` changes the meaning to greater than or equal.

16. Monotonic Stack in Practical Systems

The same idea can help reason about:

- The next higher latency measurement
- The next time traffic exceeds a threshold
- The next larger stock or metric value
- How long a service metric remains below a future value
- Span-style monitoring questions
- Comparing sequential model evaluation scores

Example:

```
Latency measurements:
```

```
[120, 110, 130, 125]
```

```
For 120 ms, the next greater latency is 130 ms.
```

```
For 110 ms, the next greater latency is 130 ms.
```

17. Pattern Comparison

Pattern	Main Idea	Common Clue	Typical Time
Two pointers	Track two positions	Sorted array, pairs, both ends	$O(n)$
Fixed window	Maintain exactly k items	Exactly k consecutive values	$O(n)$
Variable window	Expand and shrink a range	Longest/shortest continuous range	$O(n)$
Stack	Process newest first	Nested, matching, previous/next	$O(n)$
Queue	Process oldest first	Arrival order, BFS, scheduling	$O(n)$
Monotonic stack	Ordered unresolved candidates	Next greater/smaller	$O(n)$

18. Decision Guide

Use this interview thought process.

Question 1: Is the input sorted?

If yes and the problem asks for a pair:

Consider two pointers.

Question 2: Is the answer a continuous subarray or substring?

If yes:

Consider sliding window.

Question 3: Is the window size exactly k ?

If yes:

Use a fixed-size window.

Question 4: Does the window depend on a condition?

Examples:

- No duplicates
- Sum at least target
- At most k distinct values

Then:

Use a variable-size window.

Question 5: Does the problem involve nesting or the most recent unmatched item?

Then:

Use a stack.

Question 6: Must items be processed in arrival order?

Then:

Use a queue.

Question 7: Does it ask for the next or previous greater/smaller value?

Then:

Consider a monotonic stack.

19. Common Interview Pitfalls

Two Pointers

- Using two pointers on an unsorted array without justification
- Moving the wrong pointer
- Using `left <= right` when two distinct elements are required
- Returning values when the problem asks for indexes
- Forgetting whether indexes should be zero-based or one-based

Sliding Window

- Forgetting `+1` in window length
- Removing the wrong outgoing element
- Updating the answer before the window becomes valid
- Using `if` instead of `while` when shrinking
- Applying the positive-number sum pattern to arrays containing negatives

Stack

- Popping from an empty stack
- Forgetting to check whether the stack is empty at the end
- Matching against any opening bracket instead of the most recent one
- Storing values when indexes are required

Queue

- Using `list.pop(0)` instead of `deque.popleft()`

- Mixing up the front and back
- Forgetting to mark items as visited in BFS
- Adding the same work repeatedly

Monotonic Stack

- Choosing increasing instead of decreasing order
 - Confusing greater with greater-or-equal
 - Forgetting that unresolved elements keep the default result
 - Storing values when result positions require indexes
-

20. Interview Q&A

Q1. Why does two-pointer two-sum work only on a sorted array?

Sorting gives directional information.

- If the sum is too small, moving left right increases the sum.
- If the sum is too large, moving right left decreases the sum.

Without sorting, pointer movement does not provide this guarantee.

Q2. What is the difference between two pointers and sliding window?

Two pointers is a broad technique involving two positions.

Sliding window is a specialized two-pointer technique where left and right represent a continuous range.

Every sliding window uses boundaries.
Not every two-pointer problem represents a window.

Q3. Why is sliding window usually $O(n)$ despite a nested while loop?

Each element enters the window once and leaves the window at most once.

Therefore, total pointer movements are proportional to n .

Q4. When should you use a fixed-size window?

When the problem asks about exactly k consecutive items.

Examples:

- Maximum sum of k elements
 - Average of every k measurements
 - Number of failures in each fixed batch
-

Q5. When should you use a variable-size window?

When the window must expand or shrink to satisfy a condition.

Examples:

- Longest substring without duplicates
 - Shortest subarray with sum at least a target
 - Longest substring with at most k distinct characters
-

Q6. Why does balanced-parentheses validation require a stack?

Brackets are nested, so the most recently opened bracket must close first.

That is exactly LIFO behavior.

Q7. What is the difference between a stack and a queue?

A stack processes the newest item first.

LIFO

A queue processes the oldest item first.

FIFO

Q8. Why use deque for a Python queue?

`deque.popleft()` removes the first element in approximately $O(1)$ time.

`list.pop(0)` takes $O(n)$ because the remaining elements must shift.

Q9. Why is a monotonic stack solution often $O(n)$?

Every element is pushed once and popped at most once.

Therefore, the total number of stack operations is linear.

Q10. What should a monotonic stack store: values or indexes?

Usually indexes.

Indexes allow you to:

- Update the correct result position
 - Handle duplicate values
 - Calculate distances
 - Access both the value and its location
-

Final Revision Sheet

TWO POINTERS

- Usually sorted data
- Pair or comparison problem
- Move left/right based on condition
- Common complexity: $O(n)$

FIXED SLIDING WINDOW

- Exactly k continuous elements
- Add incoming value
- Remove outgoing value
- Common complexity: $O(n)$

VARIABLE SLIDING WINDOW

- Longest or shortest continuous range
- Expand right
- Shrink left while invalid or while valid
- Length = right - left + 1

STACK

- LIFO
- Nested structures
- Balanced parentheses
- Undo and next-greater problems

QUEUE

- FIFO
- Arrival-order processing
- BFS and task scheduling
- Python: `collections.deque`

MONOTONIC STACK

- Keeps unresolved values in ordered form
- Next greater or next smaller
- Store indexes
- Push once, pop once $\rightarrow O(n)$

Core Memory Rule

Sorted pair	$\rightarrow$ Two pointers
Exactly k consecutive	$\rightarrow$ Fixed sliding window
Longest/shortest range	$\rightarrow$ Variable sliding window
Nested/recent unmatched	$\rightarrow$ Stack
Arrival order	$\rightarrow$ Queue
Next greater/smaller	$\rightarrow$ Monotonic stack